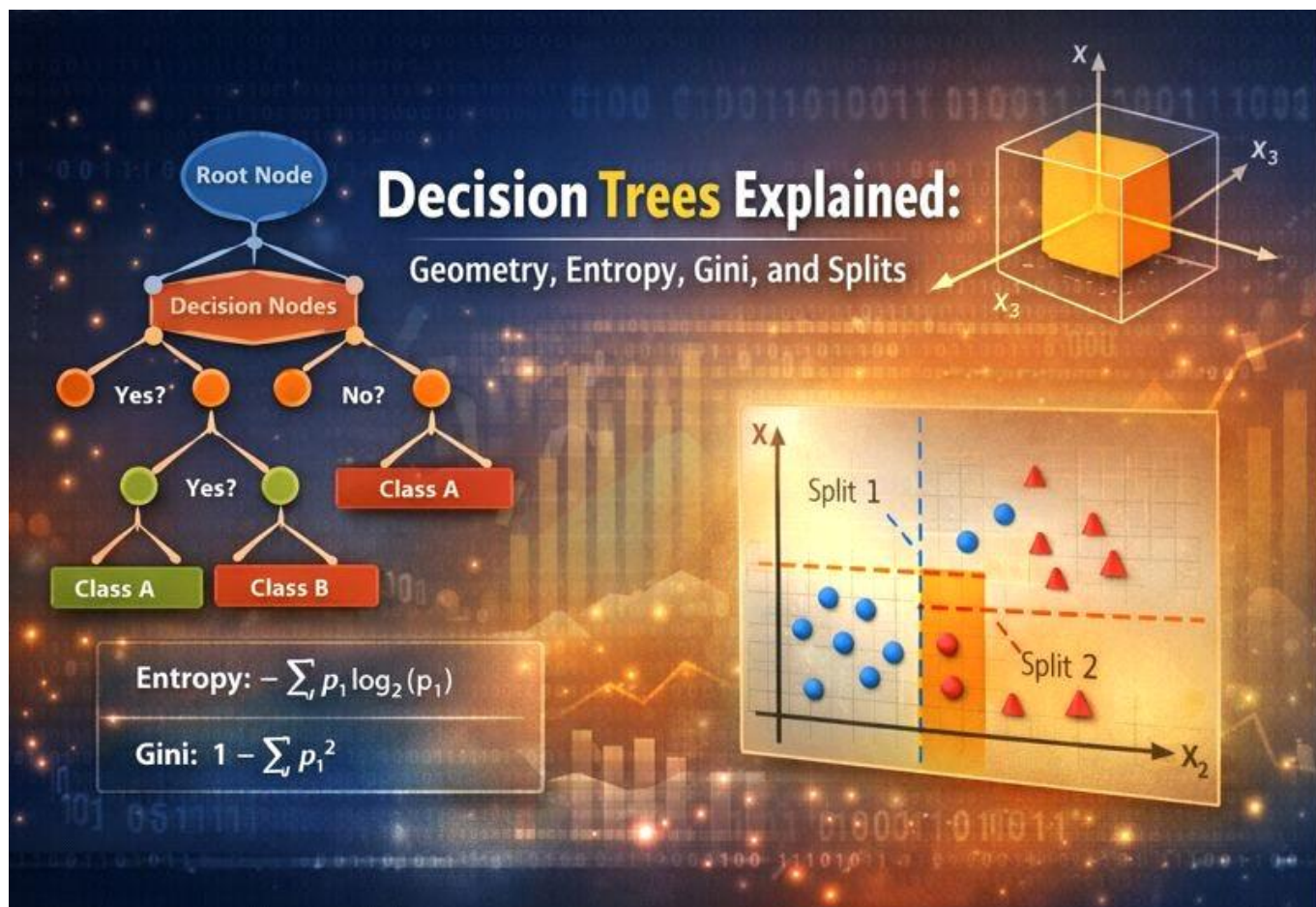


Decision Trees Explained: Geometry, Entropy, Gini, and Splits

Welcome to another post in my ongoing machine learning adventure. This blog is part of a series where I'm diving into the world of ML — learning, experimenting, and (hopefully) not getting completely lost in the algorithms! 😊 If you're interested in learning along with me, check out the whole series [HERE](#) . Or, if you're just here for this particular topic, no worries — you're in the right spot too..!

Press enter or click to view image in full size



Now we will be studying about decision trees. A decision tree is essentially a nested structure of **if-else conditions** used to classify data or predict values. It consists of a **Root Node** (the starting question), **Decision Nodes** (subsequent questions), and **Leaf Nodes** (the final prediction/output). Unlike a real tree, a decision tree in machine learning is represented upside down, with the root at the top and leaves at the bottom.

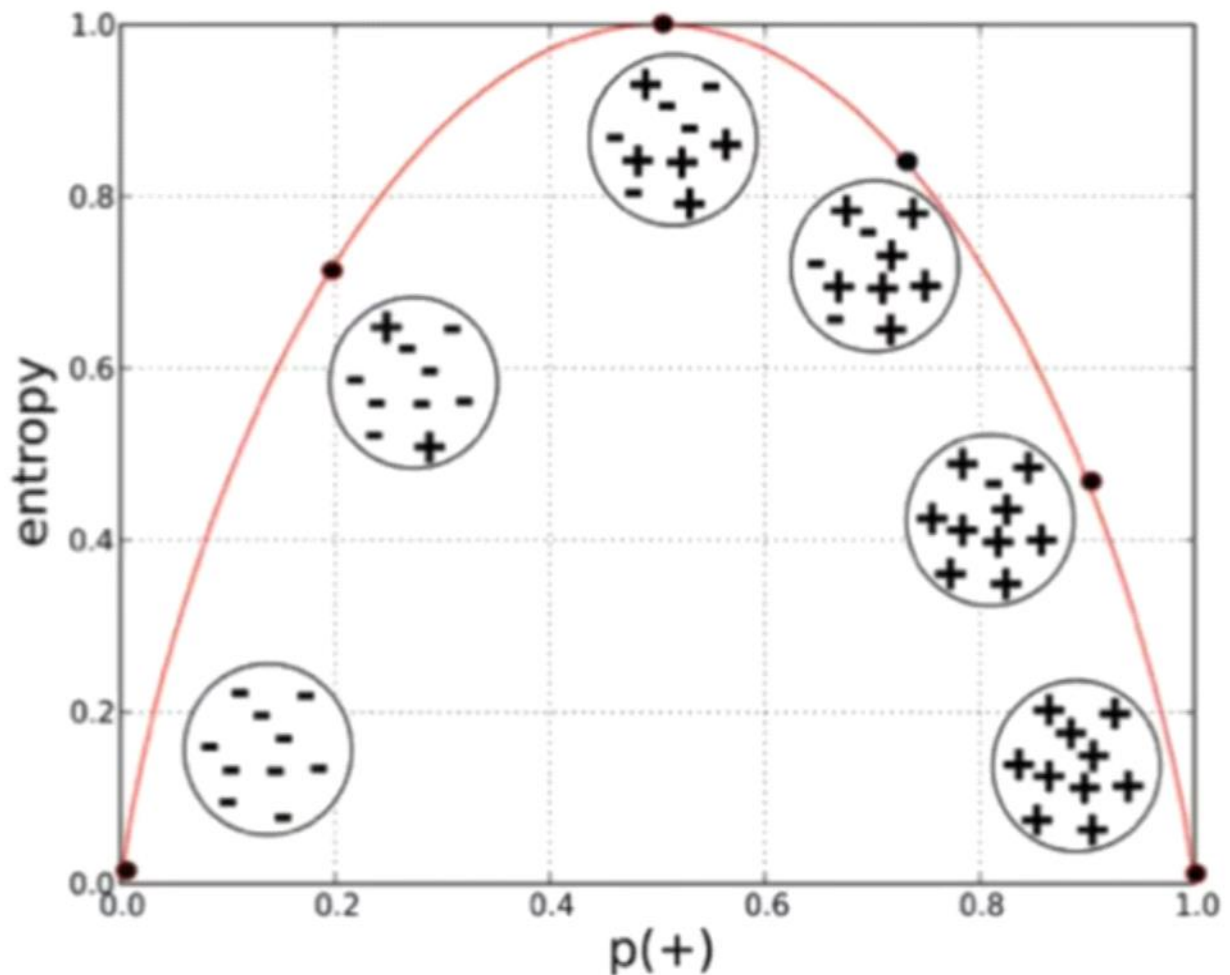
Now the question arise how our model select where to split the data or which feature should we choose to split the tree. To choose the best feature and split point, we measure **purity**. There are two ways to find this one is using **Entropy** & another using **Gini**.

Entropy (Measure of Uncertainty)

$$\text{Entropy} = - \sum p_i \log_2(p_i)$$

p_i = probability of class i .

Press enter or click to view image in full size



Entropy vs Probability Graph

If we have 2 classes only and both has equal probability the the entropy will be 1.

Gini Impurity

Gini Impurity checks how often a randomly selected sample would be mislabeled if assigned by class probability. It is computationally simple and used in tree-based classifiers.

$$\text{Gini} = 1 - \sum p_i^2$$

Where p_i is the probability of class i .

If we have 2 classes only and both has equal probability the the

entropy will be 0.5.

With the use of one of above metric we will calculate information gain.

Information Gain tells us:

Information Gain, this metric measures the quality of a split. The information gain is based on the decrease in entropy/gini after a data-set is split on an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain.

Press enter or click to view image in full size

$$\text{Information Gain} = E(\text{Parent}) - \{\text{Weighted Average}\} * E\{\text{Children}\}$$

How to find best split?

Step 1: Pick One Feature to Try a Split. Select **one feature** at a time and calculate it's entropy/gini.

Step 2: Compute **Entropy / Gini** of each child.

Step 3: Compute **weighted impurity**

Step 4: Compute Information Gain for each feature

$$\text{Information Gain} = \text{Impurity}(\text{parent}) - \text{Weighted Impurity}(\text{children})$$

Step 5: Compare Information Gain of **all splits**

Choose the split with:

Maximum Information Gain (Entropy) OR **Minimum Gini**

Impurity (Gini)

Step 6: Repeat Recursively. Apply the same steps on each child node
Although **Entropy** and **Gini Impurity** both measure node impurity in a decision tree, **Gini Impurity is often preferred in practice because it is computationally faster**, as it avoids logarithmic calculations.

However, **Entropy is sometimes used because it can produce slightly better splits on certain datasets**. Since entropy is more sensitive to changes in class probabilities, it may lead to better tree structures when class distributions are complex or highly imbalanced.

Handling Numerical Data

When a feature contains **numerical values** (such as *age*, *salary*, or *marks*), a decision tree cannot split it directly into categories. Instead, it finds an **optimal threshold** that best separates the data.

Step-by-Step Process:

1. Sort the Data

All data points are first **sorted based on the numerical feature**.

This helps the algorithm evaluate splits in an ordered manner.

2. Try All Possible Split Points

Each **unique value** in the sorted list is considered as a potential split point.

For every candidate value, the data is divided into two groups:

- i. values **less than or equal to** the threshold
- ii. values **greater than** the threshold

3. Calculate Information Gain for Each Split

For every possible threshold:

- the impurity (Entropy or Gini) of the resulting child nodes is calculated
- the **Information Gain** is computed as the reduction in impurity after the split

4. Choose the Best Threshold

The threshold that results in the **maximum Information Gain** (or minimum Gini Impurity) is selected as the final split.

This process allows decision trees to effectively handle **continuous numerical data** while maintaining simple, interpretable decision rules.

Usually Decision Tree suffer from overfitting. It occurs when a tree grows too deep, creating complex boundaries that capture “noise” rather than general patterns

Decision Trees learn simple if-else rules by recursively splitting data to maximize purity using **Entropy** or **Gini Impurity**. While they are easy

to interpret, they tend to overfit and are rarely used alone in practice. Instead, they form the foundation of powerful ensemble methods like **Random Forests** and **Gradient Boosting**.

Hyperparameters in DecisionTree

- **criterion** *{“gini”, “entropy”, “log_loss”}, default=“gini”*

These are the mathematical formulas used to measure impurity. While they usually yield similar results, Gini is often the default and slightly faster to compute.

- **max_depth** *int, default=None*

The most critical parameter. A high `max_depth` leads to overfitting, while a low `max_depth` (like 1 or 2) leads to underfitting.

- **min_samples_split** *int or float, default=2*

Sets the minimum number of samples required to split an internal node. Increasing this value prevents the tree from creating nodes for very small groups of data, which helps reduce overfitting.

- **min_samples_leaf** *int or float, default=1*

Sets the minimum number of samples required to be at a leaf node. Like the previous parameter, higher values smooth the model .

- **max_features** *int, float or {“sqrt”, “log2”}, default=None*

Limits the number of features considered for the best split at each node. Reducing this can help prevent overfitting by introducing randomness.

- **max_leaf_nodes** *int, default=None*

Limits the total number of terminal nodes (leaves) in the tree. This is another way to control the overall complexity of the model.

- **min_impurity_decrease** *float, default=0.0*

A node will be split only if the split induces a decrease of the impurity greater than or equal to this value. High values result in smaller, simpler trees

- **splitter** *{“best”, “random”}, default=“best”*

The strategy used to choose the split at each node. Supported strategies are “best” to choose the best split and “random” to choose the best random split.

Regression Trees, a type of Decision Tree used when the target variable is continuous (numerical) rather than categorical. While Linear Regression works well for linear data, it struggles with non-linear relationships. Regression Trees are better suited for datasets where the relationship between inputs and outputs is complex or non-linear. The algorithm partitions the data into smaller regions (boxes) based on feature values. For any new data point, the prediction is the **mean (average)** of the target values of the training points in that specific region.

Instead of using Entropy or Gini Impurity (as in classification), Regression Trees use the **Sum of Squared Errors (SSE)** or **Mean Squared Error (MSE)**.

1. The algorithm iterates through every possible split point for every feature.
2. For each split, it calculates the mean of the target values on both sides.
3. It then calculates the SSE (the difference between actual values and the mean, squared) for each side.
4. The split point that results in the **minimum total SSE** is chosen as the best split

Hyperparameters for RegressionTree:

- **Max Depth:** Limits how deep the tree can grow.
- **criterion**{“squared_error”, “friedman_mse”, “absolute_error”, “poisson”}, **default=“squared_error”** : The **criterion** parameter defines how a decision tree evaluates the **quality of a split**.

1. **squared_error (default):** Uses Mean Squared Error (MSE). It minimizes L2 loss by predicting the **mean** value in each leaf and is

equivalent to **variance reduction**.

2. **friedman_mse**: A modified version of MSE that uses **Friedman's improvement score**, leading to better splits in some boosting-based models.

3. **absolute_error**: Uses Mean Absolute Error (MAE). It minimizes L1 loss by predicting the **median** value in each leaf, making it more robust to outliers.

4. **poisson**: Uses reduction in **Poisson deviance**, suitable for count-based target variables.

- **Min Samples Split**: The minimum number of samples required to split an internal node.
- **Min Samples Leaf**: The minimum number of samples required to be at a leaf node.
- **Max Leaf Nodes**: Limits the total number of leaves.
- **Min Impurity Decrease**: A split is only made if it reduces the MSE by at least this amount.

We have library, **dtreeviz** is a visualization library designed to make decision trees easy to understand and interpret. Instead of showing a tree as a static structure, it visually explains **how each split is made**, including feature thresholds, class distributions, and prediction outcomes at every node. This makes it especially useful for learning, debugging, and explaining decision tree models, as it clearly shows *why* the model makes a particular decision rather than just *what* the decision is.